Find the similarity metric between two strings

Asked 8 years, 10 months ago Modified 29 days ago Viewed 322k times

409

178

How do I get the probability of a string being similar to another string in Python?

I want to get a decimal value like 0.9 (meaning 90%) etc. Preferably with standard Python and library.

e.g.

```
similar("Apple","Appel") #would have a high prob.

similar("Apple","Mango") #would have a lower prob.
```

python probability similarity metric

Share

Improve this question

Follow

edited Apr 26, 2018 at 0:59

smci

29.5k 18 109 144

asked Jun 30, 2013 at 7:35

tenstar

8,866 9 21 45

8 I don't think "probability" is quite the right term here. In any event, see stackoverflow.com/questions/682367/... – NPE Jun 30, 2013 at 7:37

2 The word you are looking for is ratio, not probability. – Inbar Rose Jun 30, 2013 at 8:21

2 Take a look at Hamming distance. – Diana Jun 30, 2013 at 8:47

3 The phrase is '*similarity metric*', but there are multiple similarity metrics (Jaccard, Cosine, Hamming, Levenshein etc.) said so you need to specify which. Specifically you want a similarity metric between strings; @hbprotoss listed several. – smci Apr 26, 2018 at 0:56

I like the "bigrams" from stackoverflow.com/questions/653157/... – MarkHu Oct 9, 2020 at 18:59

15 Answers

Sorted by: Highest score (default)

There is a built in.

750

```
from difflib import SequenceMatcher

def similar(a, b):
    return SequenceMatcher(None, a, b).ratio()
```

Using it:

```
>>> similar("Apple","Appel")
0.8
>>> similar("Apple","Mango")
0.0
```

Share Improve this answer Follow

answered Jun 30, 2013 at 8:18

Inbar Rose

39k 24 81 124

61 See this great answer comparing `SequenceMatcher` vs `python-Levenshtein` module. stackoverflow.com/questions/6690739/... – ssoler Feb 9, 2015 at 13:06

1 Interesting article and tool: chairnerd.seatgeek.com/... – DevLounge Jan 5, 2016 at 19:04

7 I would highly recommend checking out the whole difflib doc docs.python.org/2/library/difflib.html there is a `get_close_matches` built in, although i found `sorted(... key=lambda x: difflib.SequenceMatcher(None, x, search).ratio(), ...)` more reliable, with custom `sorted(... .get_matching_blocks())[-1] > min_match` checks – ThorSummoner Sep 15, 2016 at 19:51

2 @ThorSummoner brings attention to a very useful function (`get_closest_matches`). It's a convenience function that may be what you are looking for, AKA read the docs! In my particular application I was doing some basic error checking / reporting to the user providing bad input, and this answer allows me to report to them the potential matches *and* what the "similarity" was. If you don't need to display the similarity, though, definitely check out `get_closest_matches` – svenevs Sep 3, 2017 at 22:54

This worked perfectly. Simple and effective. Thankyou :) – Karthic Srinivasan May 9, 2020 at 16:39

Solution #1: Python builtin

81

use SequenceMatcher from difflib

pros: native python library, no need extra package.
cons: too limited, there are so many other good algorithms for string similarity out there.

example :

```
>>> from difflib import SequenceMatcher
>>> s = SequenceMatcher(None, "abcd", "bcde")
>>> s.ratio()
0.75
```

Solution #2: jellyfish library

its a very good library with good coverage and few issues. it supports:
- Levenshtein Distance
- Damerau-Levenshtein Distance
- Jaro Distance
- Jaro-Winkler Distance
- Match Rating Approach Comparison
- Hamming Distance

pros: easy to use, gamut of supported algorithms, tested.
cons: not native library.

example:

```
>>> import jellyfish
>>> jellyfish.levenshtein_distance(u'jellyfish', u'smellyfish')
2
>>> jellyfish.jaro_distance(u'jellyfish', u'smellyfish')
0.89629629629629637
>>> jellyfish.damerau_levenshtein_distance(u'jellyfish', u'jellyfihs')
1
```

Share

Improve this answer

Follow

edited Oct 25, 2017 at 12:34

answered Sep 8, 2017 at 22:49

Iman Mirzadeh
11.4k 1 38 42

80

I think maybe you are looking for an algorithm describing the distance between strings. Here are some you may refer to:

1. Hamming distance
2. Levenshtein distance
3. Damerau–Levenshtein distance
4. Jaro–Winkler distance

Share Improve this answer Follow

answered Jun 30, 2013 at 8:45

hbprotoss
1,354 9 26

39

`TheFuzz` is a package that implements Levenshtein distance in python, with some helper functions to help in certain situations where you may want two distinct strings to be considered identical. For example:

```
>>> fuzz.ratio("fuzzy wuzzy was a bear", "wuzzy fuzzy was a bear")
    91
>>> fuzz.token_sort_ratio("fuzzy wuzzy was a bear", "wuzzy fuzzy was a bear")
    100
```

Share

Improve this answer

Follow

edited Nov 20, 2021 at 11:41

answered Jan 18, 2017 at 22:26

BLT
2,232 1 22 33

1 updated link github.com/seatgeek/thefuzz – Ahmed Soliman Nov 19, 2021 at 10:37

Updated the link and package name, thanks. – BLT Nov 20, 2021 at 11:42

15

You can create a function like:

```
def similar(w1, w2):
    w1 = w1 + ' ' * (len(w2) - len(w1))
    w2 = w2 + ' ' * (len(w1) - len(w2))
    return sum(1 if i == j else 0 for i, j in zip(w1, w2)) / float(len(w1))
```

Share

Improve this answer

Follow

edited Sep 11, 2016 at 10:11

Ilia Kurenkov
621 5 12

answered Jun 30, 2013 at 7:41

Saullo G. P. Castro
53.2k 26 170 232

but similar('appel','apple') is higher than similar('appel','ape') – tenstar Jun 30, 2013 at 7:46

1 Your function will compare a given string against other stings. I want a way to return the string with the highest similarity ratio – answerSeeker Feb 22, 2017 at 22:55

1 @SaulloCastro, `if self.similar(search_string, item.text()) > 0.80:` works for now. Thanks, – answerSeeker Feb 22, 2017 at 23:12

Package distance includes Levenshtein distance:

10

```
import distance
distance.levenshtein("lenvestein", "levenshtein")
# 3
```

Share Improve this answer Follow

answered Apr 10, 2017 at 22:02

Enrique Pérez Herrero
3,250 2 30 32

8

Note, `difflib.SequenceMatcher` *only* finds the longest contiguous matching subsequence, this is often not what is desired, for example:

```
>>> a1 = "Apple"
>>> a2 = "Appel"
>>> a1 *= 50
>>> a2 *= 50
>>> SequenceMatcher(None, a1, a2).ratio()
0.012  # very low
>>> SequenceMatcher(None, a1, a2).get_matching_blocks()
[Match(a=0, b=0, size=3), Match(a=250, b=250, size=0)]  # only the first block
is recorded
```

Finding the similarity between two strings is closely related to the concept of pairwise sequence alignment in bioinformatics. There are many dedicated libraries for this including biopython. This example implements the Needleman Wunsch algorithm:

```
>>> from Bio.Align import PairwiseAligner
>>> aligner = PairwiseAligner()
>>> aligner.score(a1, a2)
200.0
>>> aligner.algorithm
'Needleman-Wunsch'
```

Using biopython or another bioinformatics package is more flexible than any part of the python standard library since many different scoring schemes and algorithms are available. Also, you can actually get the matching sequences to visualise what is happening:

```
>>> alignment = next(aligner.align(a1, a2))
>>> alignment.score
200.0
>>> print(alignment)
Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-
```

```
Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-
Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-
Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-Apple-
|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-
|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-
|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-
|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-|||-|-
App-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-
elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-
elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-
elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-elApp-el
```

Share Improve this answer Follow

edited Dec 4, 2019 at 10:27

answered Dec 4, 2019 at 9:36

Chris_Rands
35k 12 75 106

6

The builtin `SequenceMatcher` is very slow on large input, here's how it can be done with diff-match-patch:

```
from diff_match_patch import diff_match_patch

def compute_similarity_and_diff(text1, text2):
    dmp = diff_match_patch()
    dmp.Diff_Timeout = 0.0
    diff = dmp.diff_main(text1, text2, False)

    # similarity
    common_text = sum([len(txt) for op, txt in diff if op == 0])
    text_length = max(len(text1), len(text2))
    sim = common_text / text_length

    return sim, diff
```

Share Improve this answer Follow

answered Apr 30, 2018 at 14:24

damio
5,680 3 34 53

6

BLEUscore

> BLEU, or the Bilingual Evaluation Understudy, is a score for comparing a candidate translation of text to one or more reference translations.
>
> A perfect match results in a score of 1.0, whereas a perfect mismatch results in a score of 0.0.

Although developed for translation, it can be used to evaluate text generated for a suite of natural language processing tasks.

Code:

```
import nltk
from nltk.translate import bleu
from nltk.translate.bleu_score import SmoothingFunction
smoothie = SmoothingFunction().method4

C1='Text'
C2='Best'

print('BLEUscore:',bleu([C1], C2, smoothing_function=smoothie))
```

Examples: By updating C1 and C2.

```
C1='Test' C2='Test'

BLEUscore: 1.0

C1='Test' C2='Best'

BLEUscore: 0.2326589746035907

C1='Test' C2='Text'

BLEUscore: 0.2866227639866161
```

You can also compare sentence similarity:

```
C1='It is tough.' C2='It is rough.'

BLEUscore: 0.7348889200874658

C1='It is tough.' C2='It is tough.'

BLEUscore: 1.0
```

Share Improve this answer Follow

answered Feb 15, 2021 at 11:53

Reema Q Khan
818 1 7 18

5

You can find most of the text similarity methods and how they are calculated under this link: https://github.com/luozhouyang/python-string-similarity#python-string-similarity Here some examples;

- Normalized, metric, similarity and distance
- (Normalized) similarity and distance
- Metric distances
- Shingles (n-gram) based similarity and distance
- Levenshtein
- Normalized Levenshtein
- Weighted Levenshtein
- Damerau-Levenshtein
- Optimal String Alignment
- Jaro-Winkler
- Longest Common Subsequence
- Metric Longest Common Subsequence
- N-Gram
- Shingle(n-gram) based algorithms
- Q-Gram
- Cosine similarity
- Jaccard index
- Sorensen-Dice coefficient
- Overlap coefficient (i.e.,Szymkiewicz-Simpson)

Share Improve this answer Follow

answered Apr 9, 2020 at 14:38

Mike
135 1 8

3

There are many metrics to define similarity and distance between strings as mentioned above. I will give my 5 cents by showing an example of `Jaccard similarity` with `Q-Grams` and an example with `edit distance`.

The libraries

```
from nltk.metrics.distance import jaccard_distance
from nltk.util import ngrams
from nltk.metrics.distance  import edit_distance
```

Jaccard Similarity

```
1-jaccard_distance(set(ngrams('Apple', 2)), set(ngrams('Appel', 2)))
```

and we get:

```
0.33333333333333337
```

And for the `Apple` and `Mango`

```
1-jaccard_distance(set(ngrams('Apple', 2)), set(ngrams('Mango', 2)))
```

and we get:

```
0.0
```

Edit Distance

```
edit_distance('Apple', 'Appel')
```

and we get:

```
2
```

And finally,

```
edit_distance('Apple', 'Mango')
```

and we get:

```
5
```

Cosine Similarity on Q-Grams (q=2)

Another solution is to work with the `textdistance` library. I will provide an example of `Cosine Similarity`

```
import textdistance
1-textdistance.Cosine(qval=2).distance('Apple', 'Appel')
```

and we get:

```
0.5
```

Share

Improve this answer

Follow

edited Sep 10, 2020 at 22:56

answered Sep 10, 2020 at 22:48

George Pipis

995 10 5

2

Textdistance:

TextDistance – python library for comparing distance between two or more sequences by many algorithms. It has Textdistance

- 30+ algorithms
- Pure python implementation
- Simple usage
- More than two sequences comparing
- Some algorithms have more than one implementation in one class.
- Optional numpy usage for maximum speed.

Example1:

```
import textdistance
textdistance.hamming('test', 'text')
```

Output:

1

Example2:

```
import textdistance

textdistance.hamming.normalized_similarity('test', 'text')
```

Output:

0.75

Thanks and Cheers!!!

Share Improve this answer Follow

answered Oct 19, 2020 at 19:38

DRV
510 6 17

Adding the Spacy NLP library also to the mix;

```
@profile
def main():
    str1= "Mar 31 09:08:41  The world is beautiful"
    str2= "Mar 31 19:08:42  Beautiful is the world"
    print("NLP Similarity=",nlp(str1).similarity(nlp(str2)))
    print("Diff lib similarity",SequenceMatcher(None, str1, str2).ratio())
    print("Jellyfish lib similarity",jellyfish.jaro_distance(str1, str2))

if __name__ == '__main__':

    #python3 -m spacy download en_core_web_sm
    #nlp = spacy.load("en_core_web_sm")
    nlp = spacy.load("en_core_web_md")
    main()
```

Run with Robert Kern's line_profiler

```
kernprof -l -v ./python/loganalysis/testspacy.py

NLP Similarity= 0.9999999821467294
Diff lib similarity 0.5897435897435898
Jellyfish lib similarity 0.8561253561253562
```

However the time's are revealing

```
Function: main at line 32

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    32                                           @profile
    33                                           def main():
    34         1          1.0      1.0      0.0      str1= "Mar 31 09:08:41
The world is beautiful"
    35         1          0.0      0.0      0.0      str2= "Mar 31 19:08:42
Beautiful is the world"
    36         1      43248.0  43248.0     99.1      print("NLP
Similarity=",nlp(str1).similarity(nlp(str2)))
    37         1        375.0    375.0      0.9      print("Diff lib
similarity",SequenceMatcher(None, str1, str2).ratio())
    38         1         30.0     30.0      0.1      print("Jellyfish lib
similarity",jellyfish.jaro_distance(str1, str2))
```

Share Improve this answer Follow

answered Apr 21 at 8:05

Alex Punnen
4,082 2 47 63

0

Here's what i thought of:

```
import string

def match(a,b):
    a,b = a.lower(), b.lower()
    error = 0
    for i in string.ascii_lowercase:
            error += abs(a.count(i) - b.count(i))
    total = len(a) + len(b)
    return (total-error)/total

if __name__ == "__main__":
    print(match("pple inc", "Apple Inc."))
```

Share Improve this answer Follow

answered Dec 1, 2020 at 21:22

David Emmanuel
1 2

0

- **Python3.6+=**
- **No Libuary Imported**
- **Works Well in most scenarios**

In stack overflow, when you tries to add a tag or post a question, it bring up all relevant stuff. This is so convenient and is exactly the algorithm that I am looking for. Therefore, I coded a query set similarity filter.

```
def compare(qs, ip):
    al = 2
    v = 0
    for ii, letter in enumerate(ip):
        if letter == qs[ii]:
            v += al
        else:
            ac = 0
            for jj in range(al):
                if ii - jj < 0 or ii + jj > len(qs) - 1:
                    break
                elif letter == qs[ii - jj] or letter == qs[ii + jj]:
                    ac += jj
```

```
                    break
            v += ac
    return v


def getSimilarQuerySet(queryset, inp, length):
    return [k for tt, (k, v) in enumerate(reversed(sorted({it: compare(it, inp)
for it in queryset}.items(), key=lambda item: item[1]))))[:length]



if __name__ == "__main__":
    print(compare('apple', 'mongo'))
    # 0
    print(compare('apple', 'apple'))
    # 10
    print(compare('apple', 'appel'))
    # 7
    print(compare('dude', 'ud'))
    # 1
    print(compare('dude', 'du'))
    # 4
    print(compare('dude', 'dud'))
    # 6

    print(compare('apple', 'mongo'))
    # 2
    print(compare('apple', 'appel'))
    # 8

    print(getSimilarQuerySet(
        [
            "java",
            "jquery",
            "javascript",
            "jude",
            "aja",
        ],
        "ja",
        2,
    ))
    # ['javascript', 'java']
```

Explanation

- `compare` takes two string and returns a positive integer.
- you can edit the `al` allowed variable in `compare`, it indicates how large the range we need to search through. It works like this: two strings are iterated, if same character is find at same index, then accumulator will be added to a largest value. Then, we search in the index range of `allowed`, if matched, add to the accumulator based on how far the letter is. (the further, the smaller)
- `length` indicate how many items you want as result, that is most similar to input string.

Share

Improve this answer

Follow

edited Oct 29, 2021 at 14:13

answered Oct 29, 2021 at 14:06

Weilory

1,777 9 24